MIIA

Maestría en Inteligencia Analítica para la Toma de Decisiones (Analytics)

SESIÓN 11

Deep Learning

Series Multivariadas, Predicciones de Varios Pasos, Modelos Seq2Vec vs Seq2Seq, Secuencias Largas, LSTMs, GRUs y Otros Modelos

Daniel F. Villarraga

2026-02

Lo que cubriremos hoy

Al final de esta sesión serán capaces de:

1. Distinguir entradas multivariadas de univariados en modelos recurrentes.
2. Adaptar datasets para predecir varios pasos.
3. Comparar los esquemas Seq2Vec y Seq2Seq.
4. Explicar los límites de las RNN simples con dependencias largas.
5. Describir cómo las compuertas LSTM y GRU gestionan la memoria.
6. Identificar CNN 1D y WaveNet como alternativas para secuencias.

Series Multivariadas

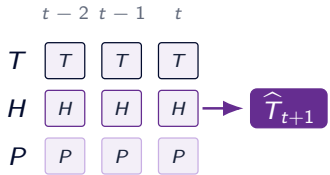
Una secuencia puede registrar varias variables en cada instante.

Serie multivariada

	$t-2$	$t-1$	t
Temp.	18	19	22
Humedad	60	62	61
Presión	1012	1011	1013

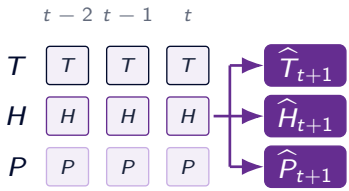
Tres variables en cada instante

Pronóstico de temperatura



Entrada: T, H, P Salida: T

Pronóstico conjunto



Entrada: T, H, P Salida: T, H, P

Ejemplo de entrada: temperatura, humedad y presión a lo largo del tiempo.

Implementación en Python

Para adaptar el pronóstico a una sola variable, basta con modificar la clase del dataset.

```
class TimeSeriesDataset_multivariate(TimeSeriesDataset):
    def __getitem__(self, idx):
        window, target = super().__getitem__(idx)
        target = target[:1]
        return window, target
```

y el input size del modelo simple de RNN que hemos usado antes:

```
    model = SimpleRNN(input_size=5, hidden_size=32, output_size=1).to(device)
```

Predicciones de Varios Pasos

Para realizar predicciones a multiples pasos se deben modificar todos los metodos de Dataset.

```
class TimeSeriesDataset_multivariate(TimeSeriesDataset):
    def __init__(self, series, window_length, output_size):
        super().__init__(series, window_length)
        self.output_size = output_size

    def __len__(self):
        return len(self.series) - self.window_length - self.output_size + 1

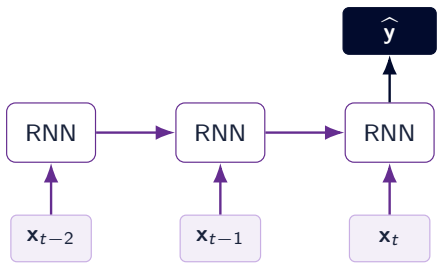
    def __getitem__(self, idx):
        end = idx + self.window_length
        window = self.series[idx : end]
        target = self.series[end : end + self.output_size, 0]
        return window, target
```

el input, y el output size del modelo simple de RNN que hemos usado antes:

```
    model = SimpleRNN(input_size=5, hidden_size=32, output_size=14).to(device)
```

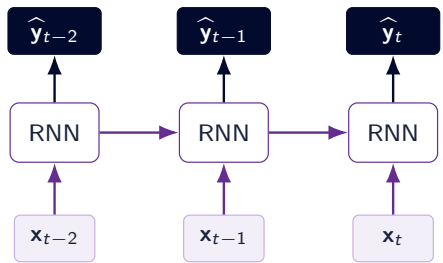
Diferencias en Tipos de Modelos: Seq2Vec y Seq2Seq

Seq2Vec



Una predicción vectorial al final de la secuencia

Seq2Seq



Una predicción vectorial por paso temporal

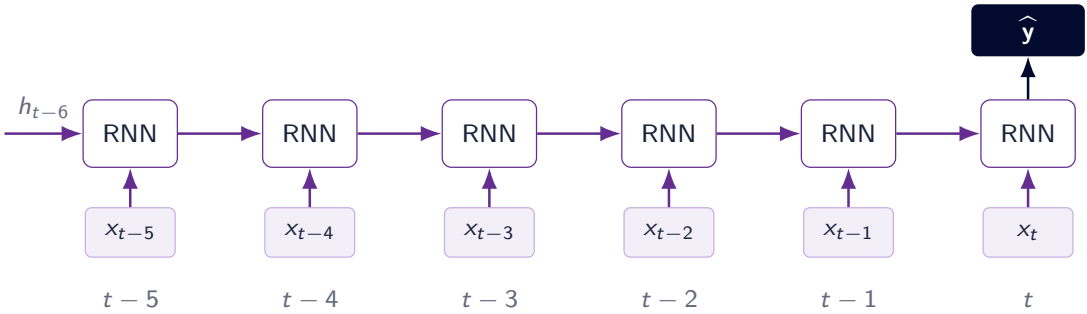
$\hat{y}$ puede ser una secuencia, un valor puntual, o un vector.

Ventajas Seq2Seq

El gradiente fluye desde cada paso:

1. Mayor rapidez, y
2. estabiliza el entrenamiento.

Dependencias Largas

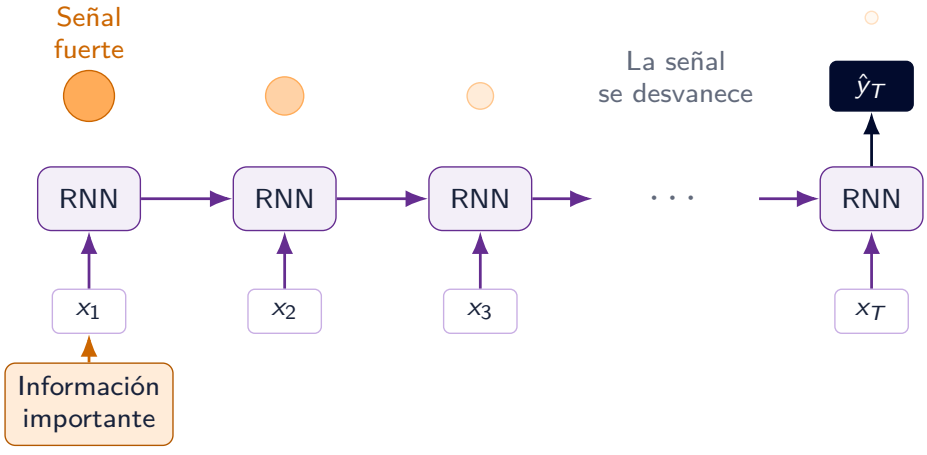


Como Fallan las RNNs Simples: Problemas de Gradiente

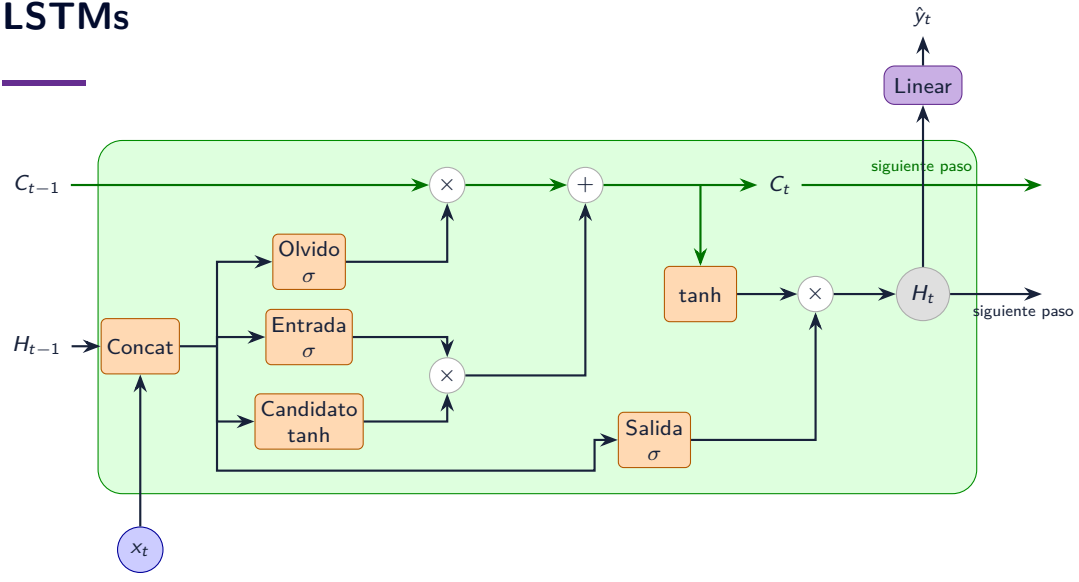
RNN Seq2Vec: 5 pasos temporales

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial \widehat{\mathbf{y}}} \frac{\partial \widehat{\mathbf{y}}}{\partial \mathbf{h}_5} \left[\frac{\partial \mathbf{h}_5}{\partial W} + \frac{\partial \mathbf{h}_5}{\partial \mathbf{h}_4} \frac{\partial \mathbf{h}_4}{\partial W} + \cdots + \frac{\partial \mathbf{h}_5}{\partial \mathbf{h}_4} \frac{\partial \mathbf{h}_4}{\partial \mathbf{h}_3} \frac{\partial \mathbf{h}_3}{\partial \mathbf{h}_2} \frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial W} \right]$$

Como Fallan las RNNs Simples: Memoria



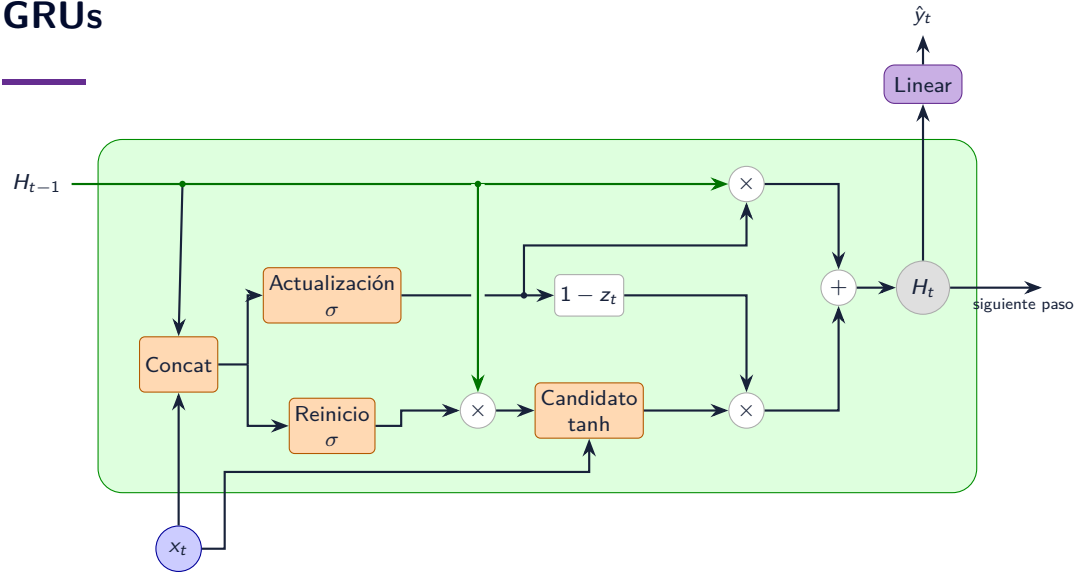
LSTMs


$\hat{y}_t$
Linear
siguiente paso
C_{t-1}
C_t
Olvido
σ
Entrada
σ
Candidato
tanh
tanh
H_t
siguiente paso
H_{t-1}
Concat
Salida
σ
x_t

Funciones en LSTMs

$f_t = \sigma\left(W_f[H_{t-1}, x_t] + b_f\right)$	Forget Gate (Olvido)
$i_t = \sigma\left(W_i[H_{t-1}, x_t] + b_i\right)$	Input Gate (Entrada)
$\widetilde{C}_t = \tanh\left(W_c[H_{t-1}, x_t] + b_c\right)$	Candidate (Candidato)
$C_t = f_t \odot C_{t-1} + i_t \odot \widetilde{C}_t$	Cell State (Estado de celda)
$o_t = \sigma\left(W_o[H_{t-1}, x_t] + b_o\right)$	Output Gate (Salida)
$H_t = o_t \odot \tanh(C_t)$	Hidden State (Estado oculto)
$\hat{y}_t = W_y H_t + b_y$	Linear (Proyección de salida)

GRUs


$\hat{y}_t$
Linear
H_{t-1}
Actualización
σ
$1 - z_t$
H_t
siguiente paso
Concat
Reinicio
σ
Candidato
tanh
x_t

Funciones en GRUs

$$r_t = \sigma\left(W_r[H_{t-1}, x_t] + b_r\right) \quad \text{Reset Gate (Reinicio)}$$

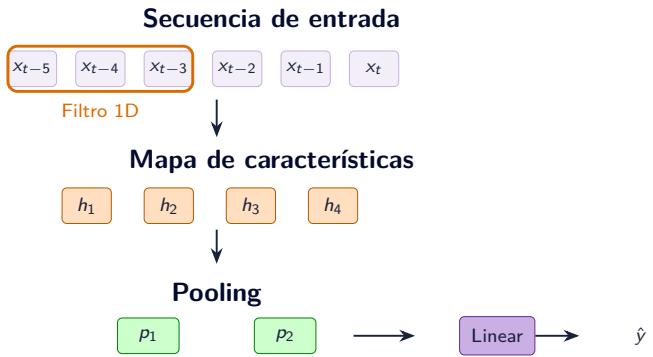
$$z_t = \sigma\left(W_z[H_{t-1}, x_t] + b_z\right) \quad \text{Update Gate (Actualización)}$$

$$\widetilde{H}_t = \tanh\left(W_h[r_t \odot H_{t-1}, x_t] + b_h\right) \quad \text{Candidate (Candidato)}$$

$$H_t = z_t \odot H_{t-1} + (1 - z_t) \odot \widetilde{H}_t \quad \text{Hidden State (Estado oculto)}$$

$$\hat{y}_t = W_y H_t + b_y \quad \text{Linear (Proyección de salida)}$$

1D CNNs para secuencias



WaveNet

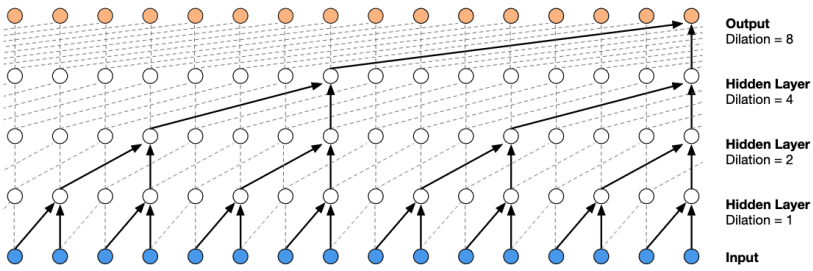


Figura: From WaveNet: A Generative Model for Raw Audio, van den Oord et al. (2016)

WaveNet

- **Convoluciones causales:** usan solo información presente y pasada, evitando *data leakage*.
- **Convoluciones dilatadas:** capturan dependencias largas sin filtros excesivamente grandes.
- **Campo receptivo exponencial:** dilataciones (1, 2, 4, 8, . . .) amplían el historial con pocas capas.